MACHINE LEARNING · UNSUPERVISED LEARNING

Hierarchical Clustering

Complete Study Notes — Concepts, Algorithms, Linkage Methods, Dendrograms & Worked Examples

WHAT'S INSIDE

- Introduction & intuition behind hierarchical clustering
- Agglomerative (bottom-up) vs Divisive (top-down) approaches
- Distance metrics and linkage criteria
- Step-by-step algorithm with worked numerical example
- Dendrograms — construction and interpretation
- Comparison with K-Means clustering
- Advantages, limitations & real-world applications
- Practice questions with answers

Study Material | Prepared for Exam Revision

1. Introduction to Hierarchical Clustering

DEFINITION

Hierarchical Clustering is an unsupervised machine learning technique used to group similar data points into clusters by building a hierarchy (tree-like structure) of clusters, rather than producing a single fixed partition of the data.

Unlike K-Means, which requires us to specify the number of clusters `k` in advance, hierarchical clustering builds a complete hierarchy of clusters and lets us decide the number of clusters later by "cutting" the hierarchy at a chosen level. This hierarchy is visually represented using a tree diagram called a **dendrogram**.

1.1 Why Hierarchical Clustering?

In many real-world problems, data has a natural multi-level structure. For example, species can be grouped into genus, family, order, and so on. Hierarchical clustering captures this nested structure naturally, which a flat clustering method like K-Means cannot do.

KEY IDEA

Instead of asking "into how many groups should I split the data?", hierarchical clustering asks "what does the entire nested structure of groupings look like?" The number of clusters becomes a choice made *after* seeing this structure, not before.

1.2 Two Main Approaches

Approach	Strategy	Direction
Agglomerative (bottom-up)	Start with every point as its own cluster; repeatedly merge the closest pair of clusters until one cluster remains.	Merging (small → large)
Divisive (top-down)	Start with all points in one cluster; repeatedly split clusters until every point is its own cluster.	Splitting (large → small)

Agglomerative clustering is far more common in practice and in exams, because it is computationally simpler and more intuitive. Divisive clustering is conceptually the reverse but is computationally expensive since it must consider all possible ways to split a cluster at each step.

2. The Agglomerative Algorithm

2.1 Step-by-Step Procedure

1. Treat each of the `n` data points as a single cluster, giving `n` clusters initially.
2. Compute the distance (or similarity) between every pair of clusters and store it in a **distance matrix**.
3. Find the two clusters that are closest to each other and merge them into a single cluster.
4. Update the distance matrix to reflect the distance between this new cluster and all remaining clusters.
5. Repeat steps 3 and 4 until only one cluster containing all data points remains.
6. Build a dendrogram recording the order and distance at which every merge happened.

> **EXAM TIP**
> Agglomerative clustering performs exactly `n − 1` merges to go from n clusters down to 1 cluster. This is a frequently asked numerical fact.

2.2 Distance Metrics Between Points

Before clusters can be merged, we need a way to measure distance between individual data points. The most common metrics are:

Metric	Formula	Notes
Euclidean Distance	$\sqrt{\Sigma(x_i - y_i)^2}$	Most widely used; sensitive to scale
Manhattan Distance	$\Sigma\lvert x_i - y_i\rvert$	Sum of absolute differences; robust to outliers
Cosine Distance	$1 - (x \cdot y)/(\lVert x\rVert \lVert y\rVert)$	Good for text/high-dimensional sparse data
Minkowski Distance	$(\Sigma\lvert x_i - y_i\rvert^p)^{(1/p)}$	Generalization; p=1 → Manhattan, p=2 → Euclidean

2.3 Linkage Criteria (How to Measure Distance Between Clusters)

Once clusters contain more than one point, we need a rule — called a **linkage criterion** — to decide the distance between two clusters (not just two points). This choice strongly affects the shape of the resulting clusters.

(a) Single Linkage (Minimum Linkage)

$$d(A, B) = \min \{ d(a, b) : a \in A, b \in B \}$$

Distance between two clusters is the distance between their **closest** pair of points. Tends to produce long, "chained" clusters and is sensitive to noise.

(b) Complete Linkage (Maximum Linkage)

$$d(A, B) = \max \{ d(a, b) : a \in A, b \in B \}$$

Distance between two clusters is the distance between their **farthest** pair of points. Produces more compact, evenly-sized clusters; less sensitive to noise than single linkage.

(c) Average Linkage

$$d(A, B) = (1 / |A||B|) \times \Sigma\, \Sigma\, d(a, b)$$

The average distance between every pair of points across the two clusters. A balanced compromise between single and complete linkage.

(d) Centroid Linkage

$$d(A, B) = d(\text{centroid of A, centroid of B})$$

Distance between the centroids (means) of the two clusters.

(e) Ward's Method

Merges the pair of clusters that leads to the **minimum increase in total within-cluster variance (sum of squared errors)**. It tends to create clusters of similar size and is one of the most popular choices in practice because it often gives visually clean, well-balanced clusters.

COMPARISON AT A GLANCE

Single linkage → can produce elongated "chains" (chaining effect).
Complete linkage → produces tight, compact, round clusters.
Average linkage → balances between the two extremes.
Ward's method → minimizes variance; very popular default choice.

3. Worked Numerical Example

Consider 5 one-dimensional data points: `A = 2, B = 4, C = 10, D = 12, E = 13`. We will perform agglomerative clustering using **single linkage** with Euclidean (absolute) distance.

3.1 Step 1: Initial Distance Matrix

	A (2)	B (4)	C (10)	D (12)	E (13)
A	0	2	8	10	11
B	2	0	6	8	9
C	8	6	0	2	3
D	10	8	2	0	1
E	11	9	3	1	0

3.2 Step 2: Successive Merges

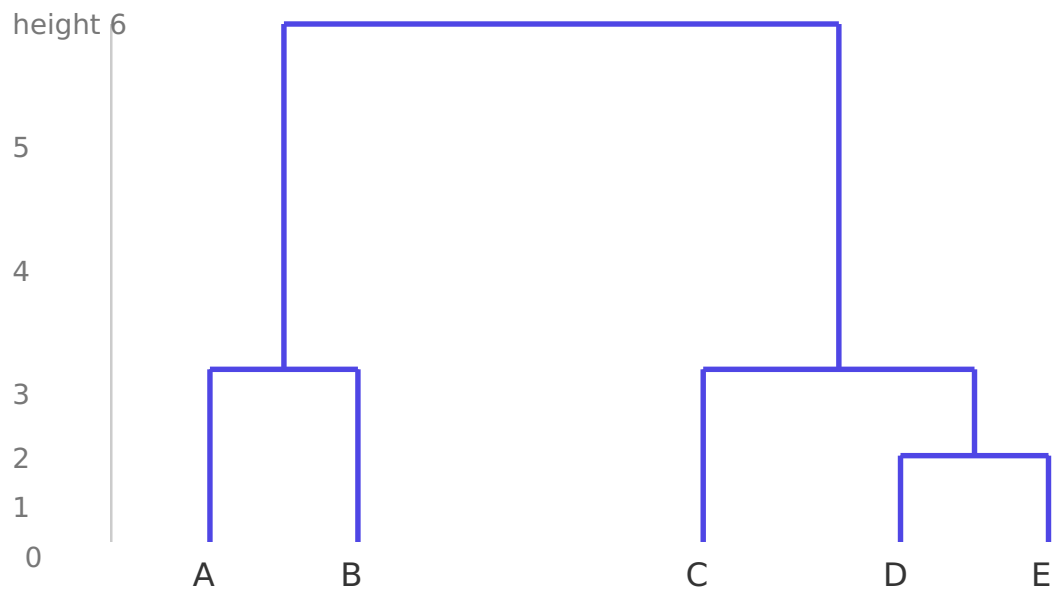
Merge Step	Closest Pair	Distance	New Cluster
1	D, E	1	{D, E}
2	A, B	2	{A, B}
3	C, {D,E}	2	{C, D, E}
4	{A,B}, {C,D,E}	6	{A, B, C, D, E}

At each step, the distance between a merged cluster and a remaining point/cluster uses the single-linkage rule (minimum distance). For example, in step 3, distance({D,E}, C) = min(d(D,C), d(E,C)) = min(2, 3) = 2.

EXAM TIP

Always show the updated distance matrix after each merge if the question asks for full working — examiners award marks for showing the matrix update process, not just the final dendrogram.

3.3 Resulting Dendrogram



Dendrogram for the worked example using single linkage. The vertical axis shows the merge distance (height).

4. Understanding Dendrograms

DEFINITION

A dendrogram is a tree-like diagram that records the sequence and distances of merges (or splits) performed during hierarchical clustering. The leaves represent individual data points, and each internal node represents a merge of two clusters at a particular distance (height).

4.1 Reading a Dendrogram

- **Leaves (bottom):** individual data points.
- **Height of a merge (y-axis):** the distance at which two clusters were combined. Lower merges = more similar clusters.
- **Horizontal cut:** drawing a horizontal line across the dendrogram at some height and counting the number of vertical lines it crosses gives the number of clusters at that distance threshold.

4.2 Choosing the Number of Clusters

The most common technique is to look for the **largest vertical gap** (longest line not crossed by a horizontal cut) in the dendrogram — cutting through this gap gives the most natural and stable clustering, since it represents merges that occurred at a noticeably greater distance than nearby merges.

EXAM TIP

A common exam question is: "Given this dendrogram, how many clusters do you get if you cut at height h?" Simply draw a horizontal line at height h and count the number of vertical lines it intersects — that count is the number of clusters.

5. Agglomerative vs Divisive Clustering

Aspect	Agglomerative	Divisive
Direction	Bottom-up (merge)	Top-down (split)
Starting Point	n singleton clusters	1 cluster with all points
Computational Cost	$O(n^2 \log n)$ typically	$O(2^n)$ in worst case — much higher
Common Use	Very common, widely implemented	Rare; used in specialized cases
Decision Made	Which 2 clusters to merge	How to split 1 cluster into 2

6. Hierarchical Clustering vs K-Means

Aspect	Hierarchical Clustering	K-Means
Number of clusters (k)	Not required in advance; decided after viewing dendrogram	Must be specified in advance
Output	A complete hierarchy / dendrogram	A single flat partition
Reproducibility	Deterministic (same result every run)	Depends on random centroid initialization
Scalability	Poor for very large datasets ($O(n^2)$ or worse memory/time)	Scales well to large datasets
Cluster shape	Can capture nested, non-spherical structures (depending on linkage)	Assumes roughly spherical, equally-sized clusters
Sensitivity to outliers	Single linkage is sensitive; Ward's/complete linkage less so	Sensitive — outliers distort centroids

7. Advantages and Limitations

ADVANTAGES

- No need to pre-specify the number of clusters
- Produces an interpretable dendrogram showing nested structure
- Deterministic — no randomness in results, unlike K-Means
- Works with any valid distance/similarity measure
- Can reveal hierarchical relationships in the data (e.g. taxonomies)

LIMITATIONS

- Computationally expensive: typically $O(n^2)$ or $O(n^3)$ time/space
- Not suitable for very large datasets
- Once a merge/split is made, it cannot be undone (greedy, irreversible)
- Sensitive to noise and outliers (especially single linkage)
- Choice of linkage method/distance metric heavily affects results

8. Real-World Applications

- **Bioinformatics:** constructing phylogenetic trees to show evolutionary relationships between species.
- **Document/Text Clustering:** organizing documents into topic hierarchies.
- **Customer Segmentation:** grouping customers based on purchasing behavior at varying levels of granularity.
- **Image Segmentation:** grouping pixels with similar properties into regions.
- **Social Network Analysis:** detecting nested community structures.
- **Gene Expression Analysis:** grouping genes with similar expression patterns across experiments.

9. Summary & Quick Revision

Key Takeaways

- Hierarchical clustering builds a tree (dendrogram) of nested clusters instead of one fixed partition.
- Two types: **Agglomerative** (bottom-up, common) and **Divisive** (top-down, rare).
- Agglomerative clustering performs **n−1 merges** for n data points.
- Linkage criteria — Single, Complete, Average, Centroid, Ward's — determine how cluster-to-cluster distance is computed and shape the resulting clusters differently.
- The dendrogram height represents merge distance; cutting it horizontally at a chosen height yields the final clusters.
- Unlike K-Means, no need to predefine `k`; results are deterministic but computationally heavier ($O(n^2)$ or more).
- Single linkage → chaining effect; Complete linkage → compact clusters; Ward's method → minimizes variance, very popular.

9.1 Practice Questions

Q1. How many merge operations occur in agglomerative clustering for 8 data points?

A1. $n - 1 = 8 - 1 = 7$ merges.

Q2. Which linkage method is most prone to the "chaining effect"?

A2. Single linkage, because it merges clusters based only on their nearest points, allowing long chains of points to be linked together.

Q3. If a dendrogram is cut at a height where 3 vertical lines are crossed, how many clusters result?

A3. 3 clusters.

Q4. Why is divisive clustering used less often than agglomerative clustering?

A4. Because at each step it must evaluate all possible ways of splitting a cluster into two, which is computationally very expensive (exponential in the worst case), unlike agglomerative clustering's simpler pairwise merge.

Q5. Which linkage method minimizes the increase in total within-cluster variance at each merge?

A5. Ward's method.